

Sequence Modeling and RNNs

Overview

- evaluate and critique techniques for modeling sequential data with N-grams and Recurrent Neural Networks (RNNs)
- identify and describe the probabilistic basis of Language Models

Async

**Modeling
Sequential Data**

- examples of sequence modeling
 - weather forecasting
 - stock market trends
 - autocomplete for texting
 - genetic sequencing
 - speech recognition
 - video frame projection
 - music composition
- sequences can be thought about as a list and can span many scopes and formats
 - bytes
 - letters (words)
 - words (sentences)
 - sentences (documents)
 - frames (video)
 - amino-acids (genetic code)

- the goal of sequence modeling is to compute the probability of a sequence of items

$$P(X) = P(x_1, x_2, x_3, \dots, x_n)$$

- what is the next item in the sequence?

$$P(x_5 | x_1, x_2, x_3, x_4)$$

- recall the chain rule:
 - definition of conditional probabilities

$$P(A, B) = P(A)P(B|A)$$

- with > 2 variables

$$P(A, B, C, D) = P(A)P(B|A)P(C|B, A)P(D|A, B, C)$$

- history-based models

$$P(x_1 \dots x_n) = P(x_1)P(x_2|x_1)P(x_3 \dots x_n|x_1, x_2)$$

$$= \prod_{i=1}^n P(x_i | x_1, x_2 \dots x_{i-1})$$

implication: *each token is dependent on all previous tokens*

- this presents a complexity issue as the number of parameters scales with length of sequence

Markov Models

- the complexity issue can be mitigated by limiting the history with Markov Assumption
 - state is independent of history given previous state

$$P(x_i | x_1, x_2 \dots x_{i-1}) = P(x_i | x_{i-1})$$

$$P(x_1 \dots x_n) = \prod_{i=1}^n P(x_i | \overbrace{x_1, x_2 \dots x_{i-1}}^{\text{ignore older history}}) = \prod_{i=1}^n P(x_i | x_{i-1})$$

ignore older history

- first order MM:
 - maintain history of last state
 - bigram model
- second order MM:
 - maintain history of last two states
 - trigram model
- Examples:
 - Bigram model trained on "The Legend of Sleepy Hollow"
 - "But it were grunting in former times of the battle in the quietest places which last was found favor in with snow which he heard in a knot of doors of the fence Ichabod stole forth now came to look behind"
 - Trigram model
 - "As Ichabod jogged slowly on his haunches and unskillful rider that he was according to the lot of a snowy night, with what wistful look did he shrink with curdling awe at the mention of the screech owl"
 - Five-gram model
 - "About two hundred yards from the tree a small brook crossed the road and ran into a marshy and thickly wooded glen known by the name of the Headless Horseman of Sleepy Hollow"
- clearly the quality increases quickly with the 5-gram generation being completely intelligible

- limitations of Markov models
 - finite context
 - Markov models are constrained by their fixed order, which limits their ability to capture long-term dependencies
 - lack of memory
 - do not possess memory of past states or events
 - stationarity
 - assume that the data-generating processes remain fixed over time

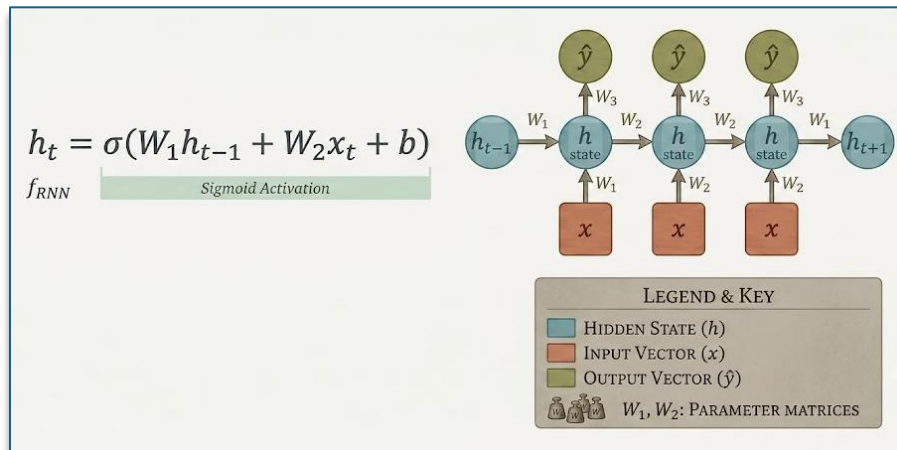
RNNs: Form and Parameters

- Recurrent Neural Networks recursively update hidden state to handle varying context sizes
- often context changes, branches, and runs in parallel, with articles near the end of the prompt referencing information presented anywhere in the earlier text up to the very beginning
- the notion of state in the RNN can be expressed:

$$h_t = \sigma(W_1 h_{t-1} + W_2 x_t + b)$$



Gemini 3.5



Gemini 3.5

- recursively combine new input with old state to handle varying context sizes
- λ is a transformational function to get output from state:
 - binary classification
 - sigmoid
 - if desired output = $\{+1, 1\}$
 - multi-class classification
 - softmax
 - if desired output = $\{1, 2, 3 \dots K\}$

RNNs in Action

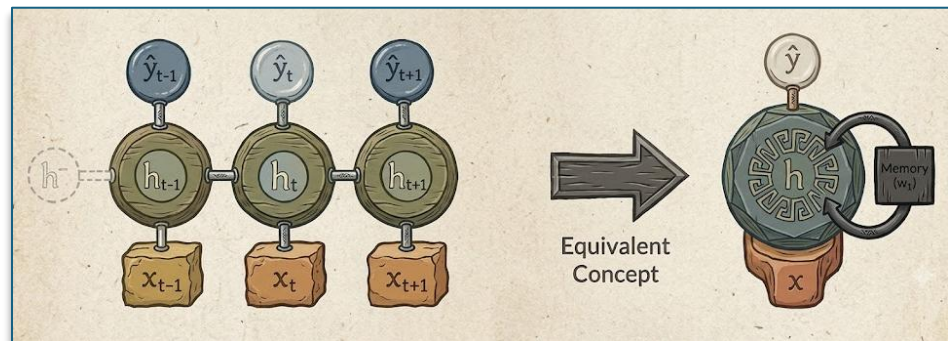
- generating sequences using RNNs
 - assume we currently have observation x_1 and want to predict x_2
 - we compute hidden state h_1 first

$$h_1 = \sigma(W_1 h_0 + W_2 x_1 + b)$$
 - then, predict next token $\hat{x}_2$ by:
 - first, computing $W_3 h_1$ and applying λ to get a distribution $\hat{y}_1$ (e.g., over a vocabulary)

- then, sampling $\hat{x}_2$ from the distribution $\hat{y}_1$

$$\hat{x}_2 \sim \hat{y}_1 = \lambda(W_3 h_1)$$

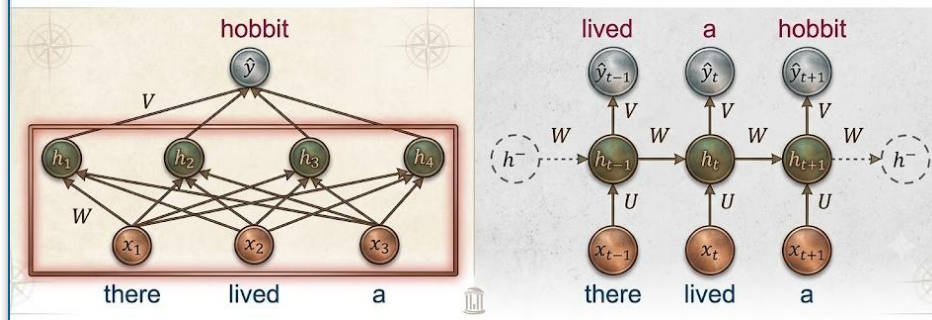
- an RNNs is a type of NN that has a recurrence structure
 - each hidden unit (h_t)
 - encodes information from previous unit (h_{t-1})
 - which in turn encodes information from its previous unit (h_{t-2})
 - and so on...
 - can model long range dependencies



Gemini 3.5

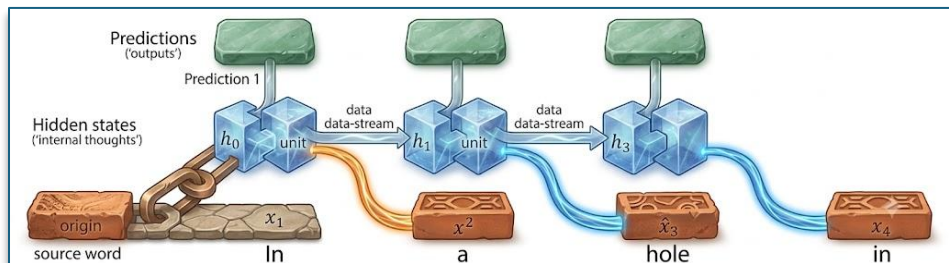
- FNNs vs RNNs
 - FNNs:
 - number of inputs is fixed (fixed context)
 - each hidden unit is dependent on all inputs but not on other hidden units
 - RNNs:
 - unlimited number of inputs (flexible context and long-range dependencies)
 - each hidden unit is dependent only on the previous hidden unit and current input

FNNs vs. RNNs



Gemini 3.5

- RNNs are Auto-Regressive Models
 - an ARM predicts value at time t based on a function of the previous values at times $t - 1, t - 2$, and so on...
 - therefore, an RNN-based LM is also called an auto-regressive generation model



Gemini 3.5